5a. Overview and Goals

Martin Alfaro
PhD in Economics

Thus far, we've laid the groundwork by introducing the fundamentals of Julia. We've presented typical objects used in Julia, distinguishing between scalars and collections, and introduced core programming tools such as functions, conditions, and for-loops. At this initial stage, nonetheless, the emphasis was primarily on familiarizing with the core approaches and their syntax. Thus, we didn't delve into how the tools can be applied and combined.

Equipped with a foundational knowledge of the concepts, we can now begin exploring techniques for working with collections. In particular, Chapter 5 focuses on methods for manipulating **mutable collections**, which are collections whose elements can be modified. Our primary example will be vectors, the most common mutable collections in Julia.

Section 5b introduces **the concept of mutability** in a formal way and identifies which objects possess this property. Once we understand what it means for an object to be mutable, Section 5c establishes **a clear distinction between assignments and mutations**. The difference determines whether we're creating a new object or modifying an existing one. While the distinction may seem unimportant, it'll be crucial for high performance.

The remaining sections concentrate entirely on vectors. Section 5d shows **how to initialize vectors**, Section 5e explains **how to define subsets of elements** (called slices), and Section 5f covers selecting individual elements via **indexing**.

The final two sections deal with operations that modify vectors, giving rise to the concept of **in-place operations**. Section 5g introduces ways to perform these operations. Meanwhile, Section 5h concentrates on mutations carried out through functions. They lead to the notion of **in-place functions**, defined as functions that update the values of at least one of their arguments. These functions are easily recognized in Julia, since developers follow the convention of appending the symbol `!` to their names. For example, if we want to sort the elements of a vector `x`, we can either create a new sorted vector by writing `y = sort(x)` or directly modify `x` by writing `sort!(x)`.

As we begin to integrate these tools, it may take some time to become fully comfortable with their use. In fact, you may occasionally need to revisit the earlier sections on functions, conditions, and for-loops.